

```
require('dotenv').config();
const express = require('express');

const app = express();
app.use(express.json());

// The API key remains strictly server-side
const GROQ_API_KEY =
process.env.GROQ_API_KEY;
const GROQ_URL =
"https://api.groq.com/openai/v1/chat/comple
tions";

const systemPrompt = `
You are JARVIS, an advanced personal AI
assistant.
Creator & Boss: Mohsin Khan.
Location: Chhatrapati Sambhajinagar,
Maharashtra, India.
Always address Mohsin as "Boss".

Language Requirements:
Speak naturally in conversational Hinglish.

Domain Expertise:
1. YouTube growth, algorithms, Shorts
strategy and content creation.
2. Marvel Cinematic Universe and comics
lore.
3. Coding, programming and software
```

development.

4. Indian festivals, dates, significance and upcoming occurrences.

Behavioral Directives:

Be concise by default, but highly detailed when Boss explicitly asks for detail.

Never pretend an action was completed if it requires an external capability you lack.

Do NOT return markdown or arbitrary code blocks to the user interface.

Execution Schema:

You MUST respond strictly with a valid JSON object matching this exact schema, and nothing else:

```
{
  "type": "ACTION",
  "action": "OPEN_APP | OPEN_URL |
OPEN_SETTINGS | SEARCH_WEB |
SHOW_INFORMATION | CREATE_NOTE |
SET_REMINDER | SPEAK | NO_ACTION",
  "target": "identifier, app name, or url
(optional)",
  "parameters": { "key": "value" },
  "requiresConfirmation": boolean,
  "spokenResponse": "Your natural Hinglish
response here"
}
```

```
`;

app.post('/api/jarvis/v1/query', async
(req, res) => {
  try {
```

```

    const { userQuery, currentDateTime
} = req.body;

    const contextualPrompt = `
    SYSTEM TIME: The current date and
time is ${currentDateTime}. Never hardcode
the date.

    Boss says: "${userQuery}"
    `;

    const groqResponse = await
fetch(GROQ_URL, {
    method: 'POST',
    headers: {
        'Content-Type':
'application/json',
        'Authorization': `Bearer
${GROQ_API_KEY}`
    },
    body: JSON.stringify({
        model: "llama-3.3-70b-
versatile",
        messages: [
            { role: "system",
content: systemPrompt },
            { role: "user",
content: contextualPrompt }
        ],
        response_format: { type:
"json_object" },
        temperature: 0.7
    })
});

```

```

        if (!groqResponse.ok) {
            const errText = await
groqResponse.text();
            throw new Error(`Groq API
error: ${groqResponse.status} -
${errText}`);
        }

        const data = await
groqResponse.json();
        let responseText =
data.choices[0].message.content;

        // Sanitize potential markdown
artifacts injected by the LLM
        responseText =
responseText.replace(/```json/g,
'').replace(/```/g, '').trim();
        const parsedJson =
JSON.parse(responseText);

        res.json(parsedJson);
    } catch (error) {
        console.error("AI Core Fault:",
error);
        res.status(500).json({
            type: "ACTION",
            action: "SPEAK",
            target: null,
            parameters: {},
            requiresConfirmation: false,
            spokenResponse: "Boss, backend
proxy mein technical error aa raha hai. AI

```

```
reasoning engine unreachable hai."
    });
  }
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`JARVIS
Backend Core active on port ${PORT}`));
```